Arch Computat Methods Eng
DOI 10.1007/s11831-015-9155-y

ORIGINAL PAPER

A Comparative Study of Non-traditional Methods for Vehicle Crashworthiness and NVH Optimization

Morteza Kiani[1] · **Ali R. Yildiz**[2]

Received: 17 May 2015 / Accepted: 25 May 2015
© CIMNE, Barcelona, Spain 2015

Abstract In this paper, metamodeling and five well-known metaheuristic optimization algorithms were used to reduce the weight and improve crash and NVH attributes of a vehicle simultaneously. A high-fidelity full vehicle model is used to analyze peak acceleration, intrusion and component's internal-energy under Full-Frontal, Offset-Frontal, and Side crash scenarios as well as vehicle natural frequencies. The radial basis functions method is used to approximate the structural responses. A nonlinear surrogate-based mass minimization was formulated and solved by five different optimization algorithms under crash-vibration constraints. The performance of these algorithms is investigated and discussed.

1 Introduction

A car body structure is usually evaluated under different loading conditions and constraints such as crashworthiness, NVH (Noise, Vibration, and Harshness), durability and fatigue. Under such loading conditions, a car body structure must be designed for not only the best structural performance but also lightweighting. The traditional approach that relies on trial-and-error is time consuming and not efficient for a complex large-scale automotive structure. In recent years, the rapid development of computer technology has enabled the use of computer-based design optimization as a promising tool for the design of aircraft, naval, marine and automotive structures [1]. Nowadays, computer-based design optimization is a powerful tool for developing different approaches including simulation-based and surrogate-based methods. Although these methods search for the best solution in the design space, their process and implementation are different.

In simulation-based design optimization, one or more software tools are coupled to evaluate each generated design point directly through high fidelity simulation [e.g., finite element analysis (FEA)]. Surrogate-based from FEA builds an approximate mathematical model for each response from the FEA or other analyses at each design point. The design of experiments (DoE) is used to identify randomly generated design points before analyzing the actual structure for various responses. The collection of design points and their associated responses then will be used to generate an appropriate surrogate model for each response. Finally, all the surrogate models that represent the system's behaviors are integrated in the optimization process.

The recent literature of automotive field shows the power and efficiency of the design-based optimization approach for car body structure development. However, it could be extremely time-consuming especially when the number of design variables, load cases, and domain nonlinearity are increased in the design consideration. A part of this difficulty might be removed by improving computing facilities, yet the method of search algorithm (i.e. optimization algorithm) is the most important factor that can significantly reduce the time and convergence. Global searching, robustness, memory efficiency, and fast convergence are typical characteristics which an optimization algorithm must exhibit. With the growth of complexity of the engineering problems, demand for high efficient optimization algorithms has raised and encouraged scholars to

✉ Morteza Kiani
m.kiani.74@gmail.com; mkiani@eta.com

[1] Engineering Technology Associates Inc. (ETA), Troy, MI 48083, USA

[2] Mechanical Engineering Department, Bursa Technical University, Bursa, Turkey

Published online: 02 June 2015

develop new efficient optimization algorithms for solving typical large-scale optimization problems.

To date, different algorithms for global optimization have been successfully introduced in large-scale engineering problems. Genetic Algorithm (GA) [2], Particle Swarm Optimization (PSO) [3], Simulated Annealing [4], and Artificial Bee Colony (ABC) [5] are the typical algorithms that are used for searching the global optimum point. Besides the global optimization algorithms, local optimization algorithms such as Sequential Quadratic Programming (SQP) have been used by some automotive researchers to design car body structures [6–8]. Unlike global search algorithms, local search algorithms converge fast but may get stuck in local minima. In addition, using appropriate initial point is another important factor which influences the efficiency of the local optimization algorithms. Therefore, global optimization algorithms are more suitable for large-scale engineering problems, and local search techniques may be used within the global search algorithm just for refining the design.

Among different global optimization algorithms, a novel search algorithm called Differential Evolution algorithm (DE) introduced by Storn and Price [9] has received significant attention in the literatures due to the fast convergence speed and robustness for finding the global optimum point [9–13]. Although this algorithm has been used in many practical engineering problems, it is not yet well-adopted in automotive industry.

In this paper, a traditional crashworthiness optimization problem is augmented by inclusion of additional design criterion associated with vehicle vibration characteristics. Then the problem is solved by using a surrogate-based optimization technique and considering different optimization algorithms like Artificial Bee Colony (ABC), Differential Evolution, Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Simulated Annealing (SA). Algorithm formulations are critically revised and the performance of DE algorithm in automotive design applications is discussed in detail.

2 Optimization Algorithms

2.1 Artificial Bee Colony (ABC)

The ABC was introduced by Karaboga for mathematical optimization problems [14]. This algorithm was later developed by Karaboga and Basturk [15, 16]. It is a simple nature-inspired approach from the intelligent foraging behavior of the honeybee swarm. The ABC algorithm describes the foraging behavior, learning, memorizing and information sharing characteristics of honeybees.

The main model of foraging characteristic of honeybee swarms can be explained as the food sources and foraging. In addition, recruitment to a nectar source and abandonment of a source are the two leading modes that have been implemented in the ABC algorithm. Similar to a bee colony, the ABC algorithm has been mimicked by three groups of bees including employed, onlooker and scout bees. The colony of bees in the ABC algorithm is referred as the artificial bees and includes two groups. The first half of the artificial bees is reserved for the employed bees and the second half is considered for onlookers. The scout bees are the employed bees whose food source has been abandoned. In order to proceed with optimization, the food location is a possible solution for the problem, and the nectar amount of food source is the quality of the associated solution (fitness value) [3].

The ABC algorithm starts with a random population (i.e. $P_{initial}$) of size N for the given problem. Each solution x_i (i = 1, 2… N) is a S-dimensional vector where S is the number of design variables [3]. The population is updated in each cycle C ($C = 1, 2\ldots G$) of search process.

In the search process, an employed bee generates a modification in the solution in her memory depending on the local information. When the objective function value of the new solution is better than the previous one, the new location is kept by the employed bee instead of the old one. Otherwise, the old location is kept in the memory. In each cycle (iteration), the nectar information for the food sources and its position are shared by the employed bees with the onlooker bees on the dance area. The information provided by the employed bees is evaluated by the onlooker bee and the food source is selected with a probability of its fitness value [3]. Moreover, the onlooker bee generates a new solution and keeps the new position if the fitness value related to the new position is better than the previous one. In the ABC algorithm, an artificial onlooker bee selects a food position based on the probability value associated to the food source (P_i), which is calculated as follows:

$$P_i = \frac{F_i}{\sum_{n=1}^{N_b} F_n} \tag{1}$$

where F_i is the fitness value associated to solution x_i, N_b is the number of the food sources and is equal to the employed bees.

The ABC algorithm includes the following step to produce a candidate food position from an old position stored in memory:

$$v_{ij} = x_{ij} + r_{ij}(x_{ij} - x_{kj}) \tag{2}$$

where $j \in \{1, 2, \ldots, D\}$, $k \in \{1, 2, \ldots, BN\}$ are the randomly selected indexes "($k \neq i$), and r_{ij} is a random number between $[-1, +1]$ that controls the production of neighbor

food around x_{ij}; k serves to compare of two food positions visible by a bee [17], and D is the number of optimization parameters and BN is the number of employed bees.

Equation (5) implies that the perturbation of the position x_{ij} decreases as the difference between x_{ij} and x_{kj} decreases. Consequently, when the search step approaches the optimum design, the step length is adaptively reduced.

In the case of misleading of the food source or the ABC algorithm failed to improve the position of the food source (abandoned food), the scout bees represent a new food source. The ABC algorithm simulates this case by randomly producing a position for the food to replace the abandoned position. For example, if x_i is considered as the abandoned source and $j \in \{1, 2, \ldots, D\}$, then the scout bees discover a new food source and replace the x_i with it. This operation is expressed as follows.

$$x_i^j = x_{\min}^j + r \quad \text{and} \quad (0,1)\left(x_{\max}^j - x_{\min}^j\right) \tag{3}$$

There are three parameters that control the performance of the ABC algorithm. These parameters are the number of employed or onlooker bees (N), the value of predetermined number of cycles and the maximum number of cycles (G).

2.2 Differential Evolution Algorithm

The DE algorithm is a robust optimization technique and uses real number instead of binary transformation; therefore, it is conveniently implemented by computer programs. This algorithm can be used also in optimization problems with discrete variables [18, 19]. It handles the constraints by using penalty function method. Although the performance of DE can be highly affected by the setting of penalty factors, using co-evolution mechanism improves the overall performance of the algorithm [20].

The three important parameters of DE are mutation, crossover, and selection. Once the DE algorithm is started, a population of NP solution vectors is randomly generated. The population is improved via mutation, crossover, and selection operators. Mutation and crossover parameters are used to generate new trial vectors, and the algorithm determines which vector should be reserved for the next iteration based on the selection factor.

Mutation is defined as the operation that combines the weighted difference between two population vectors into a third vector. The mutated vector's parameters are then mixed with the parameters of another predetermined vector (i.e. the target vector) to yield the so-called trial vector. If the value of the objective function for the trial vector is better than a predetermined population member, the newly generated vector will replace the vector, which it was compared in the following generation [5].

For each target vector x_i, $G = 1, 2, 3\ldots NP$, a mutant vector is produced by

$$v_i, G+1 = x_{r1}, G + F * (x_{r2}, G - x_{r3}, G) \tag{4}$$

where $i, r1, r2, r3 \in \{1,2,\ldots, NP\}$ are randomly chosen and must be different from each other, F is the scaling factor that controls the magnitude of the differential variation of (x_{r2}, $G - x_{r3}$, G), and NP is the size of the population that remains constant during the search process.

Several types of crossover have been considered for DE; however, the most common crossover is uniform [21]. The crossover operator is used to construct a new trial vector considering the current and mutant vectors. Actually, it controls which and how many vectors should be mutated in each vector of the current population based on the following statement.

$$u_{ji,G+1} = \begin{cases} u_{ji,G+1} & \text{if} \quad (rnd_j \le CR) \quad \text{or} \quad j = rn_i \\ x_{ji,G} & \text{if} \quad (rnd_j > CR) \quad \text{and} \quad j \ne rn_i \end{cases} \tag{5}$$

where $u_{ij,\,G+1}$ is a trial vector, $j = 1, 2,\ldots, D$, $rnd_j \in [0,1]$ is a random number, CR is the crossover ratio and can vary in the interval of [0,1], $rnd_i \in (1,2, \ldots, D)$ is the randomly chosen index, and D is the number of design variables.

The final step of the DE is selection the best individual among the population that produces a better value for the objective function. When a trial vector produces a better objective function value, it is transferred into the next generation; otherwise the target vector is passed in the next generation. This operation is defined in the following statement.

$$x_{i,G+1} = \begin{cases} u_{i,G+1} & \text{if } f(u_{i,G+1}) \le f(x_{i,G}) \\ x_{i,G} & \text{otherwise} \end{cases} \tag{6}$$

2.3 Genetic Algorithm

Genetic algorithm (GA) is a metaheuristic optimization algorithm introduced by Holland and his research team in the Mid-1970s [22]. The GA algorithm is inspired by the principles of genetics and evolution, and mimics the reproduction behavior observed in biological populations. GA uses the concept of "survival the best fitness" during the search process to select or generate evolved individuals (design solutions) that are fitted to their environment (design constraint).

The evolution usually starts from a randomly generated population, and repeats in each search iteration to select the best individuals. The fitness of the generated individuals in the population is evolved, and multiple individuals are selected in each population. These individuals are recombined and mutated to generate a new population. Therefore, the desirable characteristics are evolved and maintained in the genome composition of the population through a huge number of generations.

The GA's algorithm is constructed based on three operators which are reproduction, crossover and mutation [23]. The reproduction operator uses a natural selection function to generate new individuals from the precedent individuals. This operator favors the individuals that were improved and rejects those with less reproductive potentials. The crossover operator selects pairs of strings at random and generates new pairs. Cutting the original parent strings at a randomly selected point and exchanging their tails is the simplest form of crossover. The performance of this operator is controlled by the crossover rate parameter. Finally, the mutation operator randomly mutates the value of bits in a string that is controlled by mutation rate parameter. The algorithm terminates when a maximum number of iterations is reached, a satisfactory fitness value is obtained for the current population, or the average change in the fitness value is small for new individuals.

The GA is categorized as a global search method, and can be applied to both discrete and continuous or even linear and nonlinear systems. The basic GA formulation is summarized below:

1: Choose an initial population
2: Repeat until termination is satisfied
   a. Evaluate each individual's fitness value
   b. Cross over population (typically all; if not, then the worst)
   c. Select pairs to mate from best-ranked individuals
   d. Generate new population (using selected pairs)
      i. Apply crossover operator
      ii. apply mutation operator
   e. Check termination criteria
   f. Mutation
3: Loop if not terminating

2.4 Particle Swarm Optimization

Particle Swarm Optimization (PSO) is a biologically inspired algorithm that mimics the social behaviors of bird flocking. The PSO algorithm was developed by Kennedy and Eberhart in 1995 [24]. This algorithm exhibits common computation attributes including initialization with a population of random solutions and searching for optima by updating generations [2, 25]. In PSO algorithm, each individual or solution is called particle that has a velocity and position in the problem domain, and all the particles are called as swarm. Because each particle has velocity, position of the particle always changes based on Eq. (7). The velocity is also updated by considering the particle's own experience and the experience of the particle's neighbors or the swarm and it is calculated as follows [2].

$$V_{i,k+1} = W.V_{i,k} + c_1 r_1 (P_{i,k} - X_{i,k}) + c_2 r_2 (G_k - X_{i,k}) \quad (7)$$

where $V_{i,k+1}$ is the updated velocity for particle i and represents the distance to be traveled by this particle from its current position, $X_{i,k}$ is the particle position, G_k represents the global neighborhood that defines the global best solution (g_{best}) in the whole swarm, $P_{i,k}$ represents the local neighborhood and local best position of the i-th particle. The c_1 and c_2 are the social cognitive parameters that serve to adjust the second and third terms of the velocity Eq. (7), r_1 and r_2 are the two independent random numbers between 0 and 1, and W is an inertial factor that regulates the exchange between the global and local exploration abilities of the swarm.

To help convergence of PSO, Clerc and Kennedy [26] modified the Eq. (7) by adding a constriction factor K. The Eq. (7) can be rewritten as:

$$v_{i,k+1} = K\left[W.V_{i,k} + c_1 r_1 (P_{i,k} - X_{i,k}) + c_2 r_2 (G_k - X_{i,k})\right] \quad (8)$$

$$K = \frac{2}{\left|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}\right|} \quad (9)$$

where $\varphi = c_1 + c_2, \varphi > 4$. Typically, φ is set to 4.1 and K is thus 0.729.

The PSO algorithm includes three steps to find the optimum design. First, the PSO starts with generating particle's positions and velocities, then updates the particle's velocity, and finally updates the particle's positions. To update a particle's position, the PSO algorithm adds the particle's velocity to the old information of the particle's position as Eq. (10) shows. Then the initial velocity and position vectors for the particles are generated like given in Eqs. (11–12).

$$X_{i,k+1} = X_{i,k} + V_{i,k} \quad (10)$$

$$X_{i,k} = X_{\min} + (X_{\max} - X_{\min}) \times r_1 \quad (11)$$

$$V_{i,k} = V_{\min} + (V_{\max} - V_{\min}) \times r_2 \quad (12)$$

The PSO algorithm continues until the stopping criteria are satisfied. These criteria could be considered as the maximal iteration numbers or the best particle position for all swarms.

2.5 Simulated Annealing

The concept of SA was derived from the cooling procedures of material in a heat bath (i.e. annealing) proposed by Metropolis in 1953 [27]. In the annealing process, if the melted material is cooled slowly enough, the particles

move towards an optimum energy state and form uniform crystalline structure. In contrast, the crystals take non-uniform shape even with imperfections if the melted material is cooled-down quickly (i.e. quenching). Metropolis studied solidification of the melted material as a system of particles that converges to a steady state condition when the system gradually loses the heat.

In 1982, Kirkpatrick et al. [28] took the idea of the Metropolis's concept for constructing a new optimization algorithm called Simulating Annealing. They introduced an algorithm to search stochastically for feasible solutions and converge to an optimal point based on Metropolis's annealing rule. SA is a metaheuristic technique that starts with high temperature T and any initial state (X^c). A neighborhood operator is applied to the current state X^c (having energy state $f(X^c)$) to yield new state Z (having energy state $f(z)$). During the procedure, an acceptance mechanism decides which state must be maintained as the new state. The SA algorithm cyclically repeats based on the acceptance mechanism and superior state. The acceptance mechanism checks the temperature of the systems in each cycle and selects appropriate states with the lowest temperature.

There are two different acceptance mechanisms for the SA algorithm: metropolis and logistic rules [29]. These two mechanisms are formulated in Eqs. (13) and (14).

$$p(Z) = \begin{cases} 1 & if\ f(Z) < f(X^c) \\ e^{\frac{(f(X^c)-f(Z))}{T}} & if\ f(Z) \geq f(X^c) \end{cases} \tag{13}$$

$$p(Z) = 1 - \frac{1}{1 + e^{\frac{(f(X^c)-f(Z))}{T}}} \tag{14}$$

Cooling scheme is an effective parameter that influences the performance of the SA algorithm. It was originally proposed by Kirkpatrick [27] and is adopted by considering a cooling rate α. The cooling scheme controls the temperature decrement through fixing a minimum number of transitions. Several cooling schemes have been presented in literature such as linear, logarithmic, geometric, and Lundy and Mees that are represented by Eqs. (15–18), respectively [21, 30–32].

$$T_{k+1} = T_k - \frac{T_{\max} - T_{\min}}{n-1} \tag{15}$$

$$T_{k+1} = \frac{T_k}{1 + \ln(k)} \tag{16}$$

$$T_{k+1} = T_k \times \left(\frac{T_{\min}}{T_{\max}}\right)^{\frac{1}{n-1}} \tag{17}$$

$$T_{k+1} = \frac{T_k}{1 + \frac{T_k \times (T_{\max} - T_{\min})}{(n-1) \times T_{\max} \times T_{\min}}} \tag{18}$$

where k is subscript for temperature at each iteration and $k \in \{1, 2, \ldots, n\}$, and T_k and T_{k+1} are the system temperature at iterations k and $k + 1$.

If a stationary distribution is obtained at each temperature, convergence to global optimum is guaranteed [33]. However, it is usually leads to excessive long runs, hence users usually apply faster cooling schemes at fixed number of iteration [34]. The SA algorithm continues until a specified number of iteration (n) is reached or the system meets a quasi-equilibrium state. The pseudo-code of SA algorithm is as follows:

1: generate initial solution X^c
2: get the initial Temp $T > 0$
3: while $k \leq n$ do
4: while stopping criteria not met do
5: pick random neighbor $X^k \in X^c$
6: compute $\Delta = f(X^k) - f(X^c)$
7: if $\Delta \leq 0$, set $X^c = X^k$
8: if $\Delta > 0$, set $X^k = X^c$ with probability $e^{-\Delta/T}$
9: reduce temperature by using cooling scheme (e.g. $T_{k+1} = T_k/(1 + ln(k))$)
10: return X^c

3 Definition of the Optimization Problem

The design problem is aimed at reducing the overall weight of the 1996 Dodge Neon by focusing on a group of structural components that affects both crashworthiness and vibration attributes, Fig. 1a.

The following sections present structural responses used for this study, extracting the responses, and response approximation.

3.1 Crash Analysis and Associated Responses

As Fig. 1b shows, a full-scale crash finite element (FE) model was used for crashworthiness analysis. This FE model was validated by National Highway Traffic Administration (NHTSA) based on actual Full Frontal Impact (FFI) test [35]. The vehicle FE model consists of 271,111 elements in 337 parts for a total of 1333 kg. Most of the material data are modeled by piecewise linear plasticity (MAT24) that were originally derived from the coupon testing. The Belytschko-Tsay formulation is used for element formulation (ELFORM 2), and the engine and drivetrain have been modeled by crude mesh.

It is also noteworthy that this crash model does not include any interior parts (seats, door panels, etc.), dummy

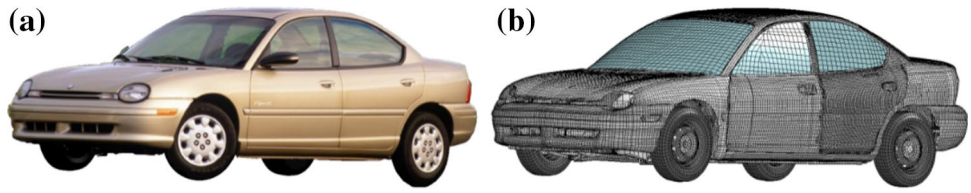

Fig. 1 **a** The real model of 1996 Dodge Neon, **b** baseline FE crash model

model and airbag; consequently, vehicle-based responses are used in lieu of the various occupant injury criteria specified in Federal Motor Vehicle Safety Standards (FMVSS). Based on FMVSS regulations, three major crash scenarios were considered and setup for the vehicle FE model. Figure 2 shows those three major crash scenarios specified by FMVSS as Full Frontal Impact (FFI), Offset Frontal Impact (OFI), and Side Impact (SI).

The FFI scenario models a frontal crash into a rigid wall at a speed of 56 km/h. The OFI has been validated at 60 km/h based on available test data [36], and is used here at a crash speed of 56 km/h to coincide with the FFI simulation at 40 % offset. For SI simulation, the stationary FE model is impacted by a moving deformable barrier (MDB) traveling at 52.5 km/h with angle of 27° relative to the vehicle. To be on the safe side that all deformation has taken place, the length of all simulations was set to 150 ms.

Depending on the scenario, one full-scale vehicle crash simulation takes anywhere up to 3 h for FFI, 12 h for OFI, and 8 h for SI on a high performance computing facility with four 6-core Intel X5660 processors and 48 GB of total RAM. Figure 3 shows the final deformation of the vehicle model in each crash scenario for the baseline FE model.

For verification of simulation results, the simulation-based acceleration curves at the left rear seat for FFI, left rear sill for OFI and at the middle of the B-Pillar on the impacted side are compared to the acceleration test data. The acceleration test data have been extracted from the accelerometers that are installed in the same locations associated with the crash scenario [35–37], Fig. 4. The

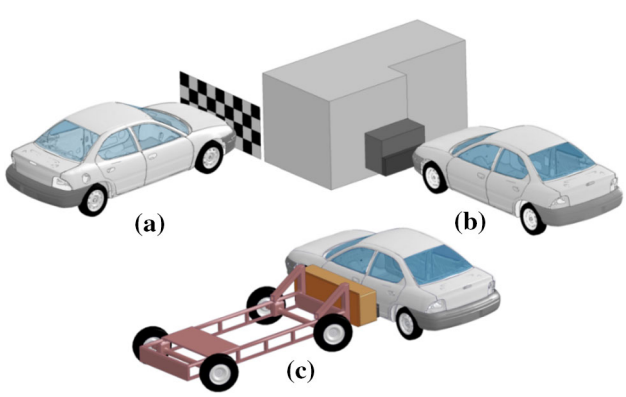

Fig. 2 Three crash scenarios: **a** FFI, **b** OFI, and **c** SI

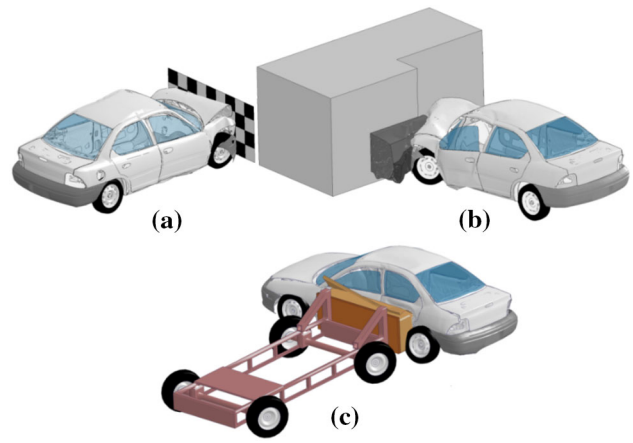

Fig. 3 Final deformed shape of the vehicle in each crash scenario: **a** FFI, **b** OFI, and **c** SI

comparison shows that the general trends of the experimental and simulation curves are the same but the peak values may differ because of filtering and methods used to capture the data. For both experimental and simulation curves, a Butterworth filter with a frequency of 60 Hz was used to remove the noise.

For each crash scenario, three different responses were considered to capture injury-based responses. The first injury-based response is the deformation of, or intrusions into, the occupant compartment that is measured by toe-board and dashboard displacement for FFI and OFI, and door intrusion in SI. The intrusion distance represents the absolute difference in the average distance measured between 20 nodes at each response location and a reference node on the opposite side of the car before and after crash.

The second injury-based response is the peak resultant acceleration measured at upper mid B-Pillar for all crash scenarios. The upper mid B-Pillar is selected to be near the approximate location of where the driver's head would be in an actual crash event [38]. To measure the peak acceleration value for each scenario, the acceleration curves for twenty nodes located at the upper B-Pillar are extracted from the simulation, and each curve was filtered by Butterworth 60 Hz. The average of the peak values of the curves set as the peak acceleration in each scenario. Figure 5 shows the locations for measuring intrusion distances and acceleration, few parts were removed to facilitate viewing.

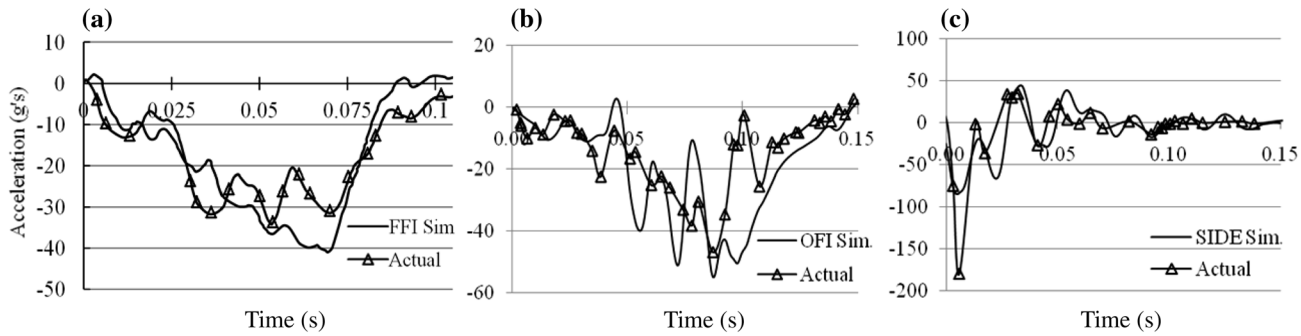

Fig. 4 Acceleration curves for **a** FFI at x-direction, **b** OFI at x-direction, and **c** SI at y-direction

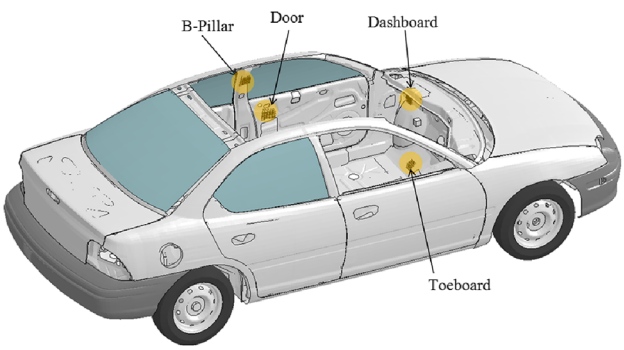

Fig. 5 Locations for measurement of intrusion distance and acceleration responses

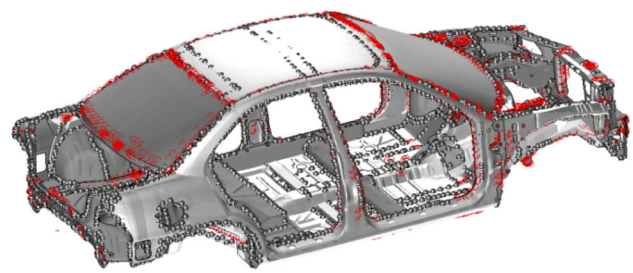

Fig. 6 BIW model of the 1996 Dodge Neon developed for vibration study

The total energy absorption by the structure is the third response considered in this study. This structural attribute reflects the potential of the car body structure to absorb more crash energy and consequently reducing the peak acceleration. The reduction of vehicle peak acceleration obviously results in higher safety of occupants during car crashes [39, 40].

3.2 Vibration Analysis and Corresponding Responses

Besides safety, Noise-Vibration-Harshness (NVH) attribute is needed to be included in the vehicle structure design [2, 6–8, 41–44]. Improving the NVH characteristics enhances the ride quality experienced by the occupants. Structural rigidity is one of the many factors that affects NVH attributes of a vehicle. The structural rigidity is usually measured by evaluating the vibration frequencies associated with numerous flexible modes.

In this study, three fundamental frequencies in bending, torsion, and combined bending-torsion are taken into account as the vibration responses. To evaluate these three mode frequencies, Body-In-White (BIW) model of Dodge Neon was developed for vibration analysis by MSC NASTRAN by Kiani co-authors [2, 6–8, 42], Fig. 6. One vibration analysis takes about 2 h by using the high performance computing system as discussed before. The vibration model of vehicle differs from the crash model in several areas such as exclusion of all moving parts such as doors, hoods, etc. Indeed, the BIW consists of all sheet metal components that are spot welded together and forms the body structure of the vehicle.

3.3 Response Approximation

Through a preliminary study [6], twenty-two components were selected such that influence the crashworthiness and vibration attributes as well as the total weight of the car. The highlighted components are shown in detail in Fig. 7. These components have a combined mass of 105.25 kg and approximately 45 % of BIW mass at 233 kg. Due to the vehicle model symmetry, these 22 components are represented by 15 wall-thickness design variables denoted by x_1 to x_{15}, and consequently both crash and vibration responses are defined with respect to these fifteen variables.

For a more comprehensive design study, one could include additional components and expand the design search space. However, a total of 22 components are considered in this study and the design space search is limited for sizing the wall thickness of the components within ±50 % of the respective baseline values.

Several surrogate techniques have been introduced and applied for response approximation. Among these techniques, Polynomial Response Surface (PRS) [2], Radial

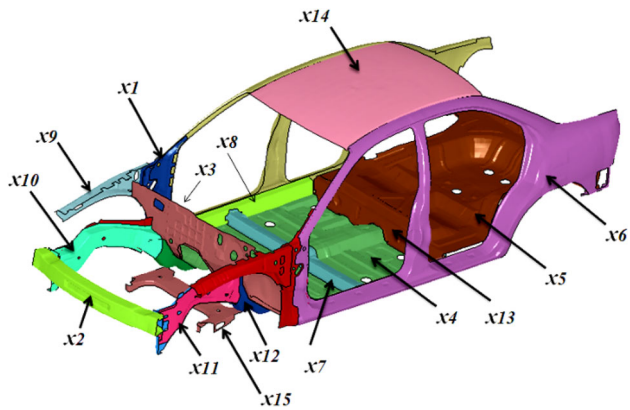

Fig. 7 Design variables and corresponding parts of 1996 Dodge Neon FE model

Basis Function (RBF) [6, 7, 42], and Kriging (KG) [45] are of importance. Based on the prior experience with RBF and its ability for accurate representation of nonlinear responses [6, 7, 42], this technique was selected for response approximation in this study. The unknown coefficients of the surrogate functions were found by using the least squares technique based on the exact function values at the selected design (training) points. These training points are usually determined from various design of experiments (DOE) techniques such as Latin Hypercube Sampling (LHS), Monte Carlo sampling, and Taguchi orthogonal array. In this study, LHS was used to sample the design space for a total of 46 training points.

RBF, originally developed for scattered multivariate data fiting, uses linear combinations of radially symmetric functions based on the Euclidean distance to approximate the relationship between the input variables (e.g., design variables) and the response of interest. RBF approximations have been shown to produce good fit to arbitrary contours of both deterministic and stochastic response functions. To accommodate the use of the tuning parameter, c, the design points are normalized in the range of 0–1; this is done by dividing each variable by the corresponding maximum value in the DOE table. Generally an RBF can be expressed as:

$$\hat{f}(y) = \sum_{k=1}^{N} \lambda_k \phi(\|y - y_k\|) = \sum_{k=1}^{N} \lambda_k \phi\left(\sqrt{(y - y_k)^T (y - y_k)}\right) \tag{19}$$

where $\hat{f}(y)$ represents the approximate function, λ_k is the coefficient associated with the kth RBF in the summation, N is the number of training points, $\boldsymbol{y}$ is the input vector of normalized variables, with $\|\boldsymbol{y} - \boldsymbol{y}_k\|$ representing the Euclidean norm or distance r from normalized design point $\boldsymbol{y}$ to the training point $\boldsymbol{y}_k$. The unknown interpolation coefficients λ_k are calculated using the least squares technique based on values of responses at the training points. The types of basis functions used in Eq. (19) include 1-thin plate spline: $\phi(r) = r^2 \ln(cr)$; 2-Gaussian: $\phi(r) = \exp(-cr^2)$; 3-multiquadric: $\phi(r) = \sqrt{r^2 + c^2}$; and 4-inverse multiquadric: $\phi(r) = 1/\sqrt{r^2 + c^2}$, with the tuning parameter defined in the range $0 \le c \le 1$.

Since $\hat{f}(\boldsymbol{y})$ value matches the exact response $f(\boldsymbol{y})$ at each training point, several arbitrary test points are used to estimate the approximation error. LHS is used to generate 46 training points, where FE models (both crash and vibration) are analyzed to extract the responses of interest. Table 1 shows the fourteen responses of interest and the corresponding RBF type and tuning parameters that minimized the average approximation error at the test points.

It is important to note that the vibration modes of the training points might switch positions relative to the baseline design when the DOE is being constructed. Therefore, Modal Assurance Criterion (MAC), based on the eigenvectors of the vibration FE model, was used to ensure that the vibration modes are identified at each training point in the DOE, and they are consistent with the similar mode frequencies of the baseline design. Indeed, a more accurate surrogate function of the natural frequencies can be obtained if the vibration modes for each training point are consistent with the baseline mode frequencies.

3.4 Optimization Setup

Through surrogate-based design optimization approach, weight minimization of the full-scale FE model for 1996 Dodge Neon was formulated as the objective function under both crash and vibration design constraints [Eq. (20)]. The wall thicknesses for twenty-two components were considered as the design variables, with each bounded to within ±50 % of the respective baseline values.

$$S.t. \begin{cases} \min & f(x) \\ g_i(x) = R_i(x) - R_i^b \le 0 & i = 1\ldots8 \\ g_i(x) = R_i^b - R_i(x) \le 0 & i = 9\ldots14 \\ 0.5x_j^b \le x_j \le 1.5x_j^b & j = 1\ldots15 \end{cases} \tag{20}$$

where $f(x)$ is the objective function defined as the total mass of the selected components; $g_{1\text{-}8}(x)$ functions are the intrusion distances of toeboard, dash board for FFI and OFI, intrusion distance of door for SI and acceleration of the B-Pillar. The $g_{9\text{–}14}(x)$ functions include the internal energy of the selected components in all three crash scenarios and also the first three fundamental frequencies of the vibration analysis. The values for $g_{1\text{–}8}(x)$ functions are required to be less or at least maintained equal to their baseline values; conversely, the $g_{9\text{–}14}(x)$ function values must be greater or equal to their baseline values.

Table 1 RBF type, tuning parameter and average error for each selected response

Response	RBF type	Tuning parameter *c*	Number of test points	Overall error percentage
FFI toe int	1	0.001	8	11.1
FFI dash int	2	0.001	5	7.3
FFI accel	1	0.001	8	8.8
FFI int eng	1	0.001	8	1.4
SI door int	2	0.001	4	4.9
SI accel	4	0.999	3	6.8
SI int eng	4	0.999	3	4.6
OFI toe int	4	0.999	5	17.9
OFI dash int	3	0.999	12	9.0
OFI accel	4	0.999	8	13.9
OFI int eng	4	0.999	2	2.9
Frq1	4	0.999	3	5.7
Frq2	4	0.999	3	5.5
Frq3	1	0.001	7	0.8

toe int toeboard intrusion, *dash int* dashboard intrusion, *accel* peak acceleration, *int eng* internal energy, *frq* frequency

4 Results and Discussion

The optimization problem stated by Eq. (20) was solved with ABC, DE, GA, PSO and SA. The performance of these algorithms was evaluated for different ranges of the total iteration number. The total iteration number was considered as 50,000. For each algorithm, the associated parameters were tuned for each range so that the highest performance and the minimum fitness value could be achieved.

In all experiments in this section, the values of the common parameters used in each algorithm such as population size and total evaluation number were chosen to be the same. Population size was 50 and the maximum evaluation number was 50,000, for all functions. The other specific parameters of algorithms are given below:

ABC Settings: Except common parameters (population number and maximum evaluation number), the basic ABC used in this study employs only one control parameter, which is called limit. A food source will not be exploited anymore and is assumed to be abandoned when limit is exceeded for the source. This means that the solution of which 'trial number' exceeds the limit value cannot be improved anymore. We defined a relation for the limit value using the dimension of the problem and the colony size:limit = (SN*D) where D is the dimension of the problem and SN is the number of food sources or employed bees.

DE Settings: In DE, F is a real constant which affects the differential variation between two solutions and set to 0.5 in our experiments. Value of crossover rate, which controls the change of the diversity of the population, was chosen to be 0.9. Probability of choosing elements of mutant vectors was chosen as 0.8.

GA Settings: In our experiments, we employed a standard GA having evaluation, fitness scaling, random selection, crossover, mutation and elite units. Multi-points crossover operation with the rate of 0.8 was employed. Mutation operation restores genetic diversity lost during the application of reproduction and crossover. Mutation rate in our experiments was 0.1. Stochastic uniform sampling technique was our selection method.

PSO Settings: Cognitive and social components are constants that can be used to change the weighting between personal and population experience, respectively. In our experiments cognitive and social components were both set to 1.8. Inertia weight, which determines how the previous velocity of the particle influences the velocity in the next iteration, was 0.6.

SA Setting: Starting and ending temperatures are 10 and 0.001 respectively where the temperature is increased exponentially for every 10 loops. For each loop, n candidates are created by mutating on the current best solution while other n candidates are created from mutating the current parent. The best of those $2n$ solutions are set as an offspring to be compared with the parent.

Table 2 Optimum mass values (kg) achieved by the five algorithms for the short-range iteration number (10,000)

	Min	Ave	Max
ABC	98.65	100.01	101.00
DE	96.92	96.96	97.04
GA	103.47	103.48	106.49
PSO	100.84	103.66	107.80
SA	99.80	102.50	109.02

Table 3 Optimum mass values (kg) achieved by the five algorithms for the mid-range iteration number (30,000)

	Min	Ave	Max
ABC	99.50	100.16	101.13
DE	96.91	96.92	96.95
GA	100.54	100.54	104.47
PSO	100.59	102.74	105.25
SA	100.17	102.28	106.35

Table 4 Optimum mass values (kg) achieved by the five algorithms for the mid-range iteration number (50,000)

	Min	Ave	Max
ABC	98.47	99.50	100.62
DE	96.90	96.91	96.93
GA	102.55	102.57	103.56
PSO	100.71	102.68	104.52
SA	100.79	103.13	106.72

Table 5 Optimum mass values (kg) achieved by the five algorithms for the long-range iteration number (100,000)

	Min	Ave	Max
ABC	98.27	99.32	99.83
DE	96.90	96.91	96.92
GA	98.75	99.23	100.19
PSO	100.21	102.61	104.38
SA	99.26	101.99	104.91

Min minimum, *Ave* average, *Max* maximum

5 Results

To evaluate the performance of each algorithm, the objective function value at the end of each cycle as well as the change in design variables were taken into account. Tables 2, 3, 4 and 5 compare the obtained objective function for each algorithm at different ranges; moreover, Fig. 9 shows the convergence of each algorithm for the mid-range iteration numbers. It is also noteworthy that the time for completing each solution by algorithms were not considered in the performance evaluation due to the code structure and computing viewpoint.

5.1 Discussion

In order to compare the five metaheuristic algorithms considered in this study, the optimized weight and the change in design variables were considered. Tables 2, 3, 4 and 5 compare the total mass determined by each algorithm for the different iteration ranges. Convergence behavior is compared in Figs. 8 and 9. Because of the inherent differences in algorithm formulations and implementation, the computation time required in the optimization will not be reported in this study.

It can be noticed that the lowest weight of the components is 96.9 kg, obtained by DE after 50,000 or 100,000 iterations. Remarkably, DE was the most robust optimizer showing just a marginal difference between best, average and worst objective function, i.e. weights. The other algorithms neither were able to reach the best weight of 96.9 kg even after 100,000 iterations nor they exhibited small deviations between best, average and worst weights. In particular, PSO and SA were the worst optimizers overall and reached a nearly optimal solution already in the 10,000–30,000 iteration ranges but could not improve their

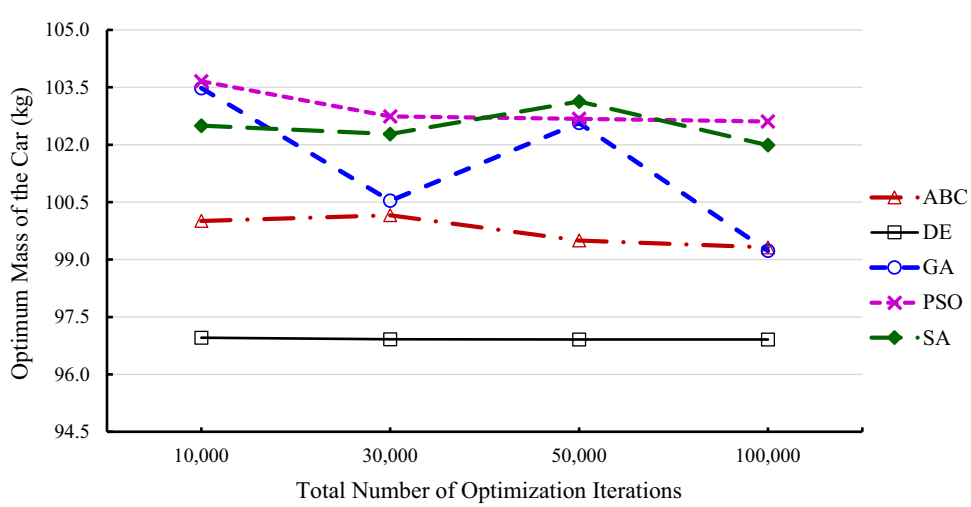

Fig. 8 Variation of optimized vehicle mass with respect to the iteration range

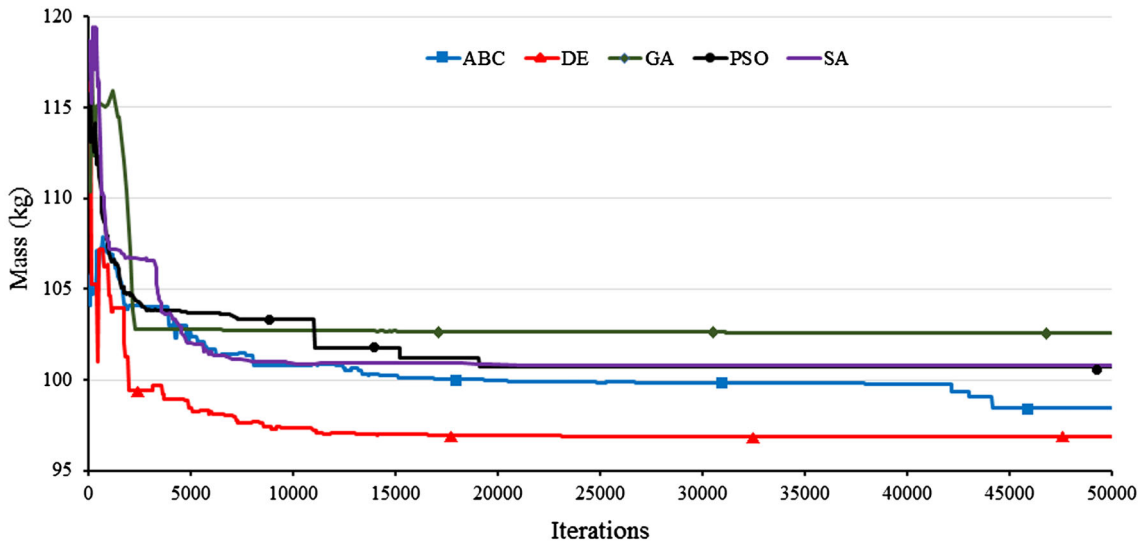

Fig. 9 Convergence behavior of the five algorithms for the second mid-range iteration number (50,000)

performance significantly in the next iteration ranges. Conversely, ABC and GA constantly improved the their performance until the very large number of iterations of 100,000, yet not as efficiently as DE.

In summary, DE has the full potentiality for being used in large-scale optimization problems of vehicle structures.

6 Conclusion

A surrogate-based optimization framework was developed in purpose of structural design optimization of a vehicle model to minimize the mass subjected to intrusion, acceleration and internal energy constraints associated with full frontal, offset frontal and side crash scenarios as well as vibration constraints defined based on three natural frequencies associated with bending and torsion modes. All the structural responses associated with crash (FFI, OFI, and Side) and vibration were approximated by surrogated models which have been generated by RBF metamodeling technique. The mass of the structure was defined as the objective function, and different global algorithms were used for minimization of the mass of the structure.

In optimization work, the efficiency of five well-known optimization algorithms which are differential evolution algorithm, artificial bee colony algorithm, particle swarm algorithm, and simulated annealing algorithm for crashworthiness and NVH optimization of a full-scale high-fidelity of vehicle model was discussed.

The relative performance of five state-of-the-art metaheuristic algorithms (i.e. differential evolution, artificial bee colony, genetic algorithm, particle swarm, and simulated annealing) was analyzed in the optimization of a full-scale high-fidelity vehicle model. It was found that DE is the best optimizer overall. This metaheuristic algorithm hence seems well suited for design optimization of large-scale automotive structures against crashworthiness coupled with NVH as well as for other real-life industrial problems.

References

1. Fang H, Solanki K, Horstemeyer MF, Rais-Rohani M (2004) Multi-impact crashworthiness optimization with full-scale finite element simulations. In: Proceedings of the 6th World congress on computational mechanics, Beijing, China
2. Kiani M, Motoyama K, Rais-Rohani M, Shiozaki H (2014) Joint stiffness analysis and optimization as a mechanism for improving the structural design and performance of a vehicle. Proc IMech Part D J Automob Eng 228(6):689–700
3. Yildiz AR, Solanki KN (2011) Multi-objective optimization of vehicle crash- worthiness using a new particle swarm based approach. Int J Adv Manuf Technol 59(1–4):367–376
4. Lee KH, Kang DH (2007) Structural optimization of an automotive door using the kriging interpolation method. Proc Inst Mech Eng Part D J Automob Eng 221(12):1525–1534
5. Yildiz AR (2013) A new hybrid artificial bee colony algorithm for robust optimal design and manufacturing. Appl Soft Comput 13(5):2906–2912
6. Kiani M, Gandikota I, Parrish A, Motoyama K, Rais-Rohani M (2013) Surrogate-based optimisation of automotive structures under multiple crash and vibration design criteria. Int J Crashworthiness 18(5):473–482
7. Kiani M, Gandikota I, Rais-Rohani M, Motoyama K (2014) Design of lightweight magnesium car body structure under crash and vibration constraints. J Magnesium Alloys 2(2):99–108
8. Gandikota I, Rais-Rohani M, DorMohammadi S, Kiani M (2014) Multilevel vehicle–dummy design optimization for mass and injury criteria minimization. Proc IMech Part D J Automob Eng 229(3):283–295

9. Storn RM, Price KV (1995) Differential evolution-a simple and efficient adaptive scheme for global optimization over continuous spaces technical report TR-95-12, International Computer Science, Berkeley, California
10. Yildiz AR (2013) Comparison of evolutionary-based optimization algorithms for structural design optimization. Eng Appl Artif Intell 26(1):327–333
11. Fender J, Duddeck F, Zimmermann M (2014) On the calibration of simplified vehicle crash models. Struct Multidiscip Optim 49(3):455–469
12. Sigmund O (2011) On the usefulness of non-gradient approaches in topology optimization. Struct Multidiscip Optim 43(5):589–596
13. Yildiz AR (2013) A new hybrid differential evolution algorithm for the selection of optimal machining parameters in milling operations. Appl Soft Comput 13(3):1561–1566
14. Karaboga D (2005) An idea based on honey bee swarm for numerical optimization. Technical report TR06, Erciyes University Press, Erciyes
15. Karaboga D, Basturk B (2007) A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. J Global Optim 39(3):459–471
16. Karaboga D, Basturk B (2008) On the performance of artificial bee colony (ABC) algorithm. Appl Soft Comput 8(1):687–697
17. Karaboga D, Ozturk C (2011) A novel clustering approach: Artificial Bee Colony (ABC) algorithm. Appl Soft Comput 11(1):652–657
18. Price KV, Storn RM (1997) Differential evolution. Dr. Dobb.s J Softw Tools Prof Program 22(4):18–24
19. Price KV, Storn RM, Lampinen JA (2005) Differential evolution a practical approach to global optimization. Springer, Berlin
20. Huang FZ, Wang L, He Q (2007) An effective co-evolutionary differential evolution for constrained optimization. Appl Math Comput 186(1):340–356
21. Storn R (2008) Differential evolution research-trends and open questions. Advances in differential evolution. Springer, Berlin, pp 1–31
22. Holland JH (1975) Adaptation in natural and artificial systems. University of Michigan Press, Ann Arbor, Michigan; re-issued by MIT Press
23. Pham DT, Karaboga D (1991) Optimum design of fuzzy logic controllers using genetic algorithms. J Syst Eng 1(2):114–118
24. Kennedy J, Eberhart RC (1995) Particle swarm optimization. Proceedings IEEE international conference on neural networks, Piscataway, 1942–1948
25. Kennedy J, Eberhart R (1997) A discrete binary version of the particle swarm algorithm. In: IEEE Systems, Man, and Cybernetics, Computational Cybernetics and Simulation, Orlando, FL, vol 5, pp 4104–4108
26. Clerc M, Kennedy J (2002) The particle swarm-explosion, stability, and convergence in a multidimensional complex space. IEEE Trans Evol Comput 6(1):58–73
27. Metropolis N, Rosenbluth AW, Rosenbluth MN, Teller AH, Teller E (1953) Equation of state calculation by fast computing machines. J Chem Phys 21:1087–1091
28. Kirkpatrick S, Gelatt CD, Vecchi MP (1983) Optimization by simulated annealing. Science 220(4598):671–680
29. Aarts E, Korst J (2002) Selected topics in simulated annealing. Essays and surveys in metaheuristics. Springer, New York, pp 1–37
30. Menon S, Gupta R (2004) Assigning cells to switches in cellular networks by incorporating a pricing mechanism into simulated annealing. IEEE Trans Syst Man Cybern Part B Cybern 34(1):558–565
31. Ceranic B, Fryer C, Baines RW (2001) An application of simulated annealing to the optimum design of reinforced concrete retaining structures. Comput Struct 79(17):1569–1581
32. Lundy M, Mees A (1986) Convergence of an annealing algorithm. Math Program 34:111–124
33. Van Laarhoven P, Aarts E (1987) Simulated annealing: theory and applications. Kluwer Academic Publishers, Norwell
34. Strenski P, Kirkpatrick S (1991) Analysis of finite length annealing schedules. Algoritmica 6:346–366
35. NCAC, Finite Element Model of Dodge Neon: Model Year 1996 Version 7. FHWA/NHTSA National Crash Analysis Center, Ashburn, VA, 2006. http://www.ncac.gwu.edu/vml/archive/ncac/vehicle/neon-0.7.pdf
36. NHTSA (1997) Frontal Barrier Forty Percent Offset Impact Test: 1996 Dodge Neon, prepared by Kargo Engineering for the U.S. Department of Transportation, Washington, DC
37. MGA (1997) Safety compliance testing for FMVSS 214. Side impact protection—passenger cars: 1997 dodge neon, prepared by MGA proving grounds for the U.S. Department of Transportation, Washington, DC
38. Hurnall J, Draheim A, Case M, Del Beato J (2003) A review of 'B'-pillar and front seat belt loads measured in ANCAP offset frontal crash tests. In: Proceedings of 18th international technical conference on the enhanced safety of vehicles, Nagoya, Japan
39. Bertocci GE, Esteireiro J, Cooper RA, Young TM, Thomas C (1999) Testing and evaluation of wheelchair caster assemblies subjected to dynamic crash loading. J Rehab Res Dev 36(1):32–41
40. Draheim A, Hurnall J, Case M, Beato JD (2005) Structural energy absorption trends in NCAP crashed vehicles. In: 19th Enhanced safety of vehicles conference, NHTSA, Washington, DC, USA, Paper 05-0317
41. Liao X, Li X, Yang Q, Li W, Zhang W (2008) A two-stage multi-objective optimisation of vehicle crashworthiness under frontal impact. Int J Crashworthiness 13(3):279–288
42. Kiani M, Shiozaki H, Motoyama K (2013) Simulation-based design optimization to develop a lightweight body-in-white structure focusing on dynamic and static stiffness. Int J Veh Des 67(3):219–236. doi:10.1504/IJVD.2015.069467
43. Duddeck F (2007) Multidisciplinary optimization of car bodies. Struct Multidisc Optim 35:375–389
44. Baskin D, Reed D, Seel T, Hunt M (2008) A case study in structural optimization of an automotive body-in-white design, SAE technical paper 2008-01-0880. SAE International, Warrendale, PA
45. Parrish A, Rais-Rohani M, Najafi A (2012) Crashworthiness optimisation of vehicle structures with magnesium alloy parts. Int J Crashworthiness 17:259–281